

AI-Powered Role-Based Candidate Screening System

Technical Project Report

Assignment: AI/ML & Backend Engineering Intern

Stack: FastAPI | React 18 | FAISS | OpenAI GPT-3.5

GitHub: [github.com/SANJAY-102/AI-powered-role-based-candidate-screening-syst](https://github.com/SANJAY-102/AI-powered-role-based-candidate-screening-system)
em.-

Date: June 2026

1. Objective

This project delivers a fully functional, end-to-end AI-powered technical interview platform. The system accepts a candidate's resume in PDF format, extracts role-specific skills and experience level, then dynamically generates adaptive interview questions grounded in authoritative ML textbooks using a Retrieval-Augmented Generation (RAG) pipeline. Each answer is scored in real time by GPT-3.5-turbo on a 1-5 scale with written feedback, and a complete evaluation report is produced at the end of every session.

2. Problem Statement

Traditional interview processes are time-consuming, inconsistent, and difficult to scale. Hiring teams spend significant time crafting role-specific questions and evaluating answers manually, which introduces bias and variable quality. For ML/AI and Backend roles, technical depth is hard to assess without domain expertise on every interview panel.

This system automates the technical screening stage: a candidate uploads their resume, the system infers their expertise level and skill set, then conducts a structured technical interview grounded only in knowledge retrieved directly from the assigned textbooks, ensuring consistency, auditability, and traceability.

3. System Architecture

3.1 High-Level Component Flow

```
Frontend (React 18 + Vite + Tailwind CSS)
| HTTP via Axios
Backend (FastAPI + SQLAlchemy + SQLite)
|           |           |
SQLite DB   FAISS Index  OpenAI GPT-3.5-turbo
(sessions, (ML textbook (question generation +
Q&A log)   embeddings)   answer scoring)
```

3.2 RAG Pipeline Steps

- Ingest: PyMuPDF extracts raw text from all ML textbook PDFs in knowledge_base/
- Chunk: Text is split into 350-word segments with 50-word overlap to preserve context
- Embed: sentence-transformers/all-MiniLM-L6-v2 produces 384-dimensional dense vectors
- Index: FAISS IndexFlatL2 stores all vectors for fast nearest-neighbour search
- Retrieve: At interview time, the skill query is embedded and top-k chunks fetched

- Generate: GPT-3.5-turbo uses retrieved context + role/difficulty prompt to form questions
- Score: GPT evaluates each answer against the same retrieved textbook source chunk

4. Knowledge Base (RAG Source Documents)

The system uses the official assignment-mandated textbooks as its sole RAG knowledge source. A total of 2,253 semantic chunks were indexed across 6 books, covering supervised learning, deep learning, NLP, probabilistic models, ensemble methods, and applied ML.

Textbook	Author	Role
Machine Learning	Tom Mitchell	AI/ML Role
The Hundred-Page ML Book	Andriy Burkov	AI/ML Role
ML for Absolute Beginners	Oliver Theobald	AI/ML Role
Master ML Algorithms	Jason Brownlee	Data Science Role
Pattern Recognition & ML	Christopher Bishop	Advanced/Theoretical
AI, ML & Deep Learning	Wani et al.	Advanced/Theoretical

Total characters extracted: approximately 4,035,000 across all six books. 2,253 vector embeddings of dimension 384 were generated in under 15 seconds on CPU.

5. Features Implemented

- Resume PDF parsing with PyMuPDF: skills, technologies, and experience level extraction
- Role-based session creation: AI/ML Engineer, Backend Engineer, Data Scientist
- FAISS semantic retrieval: top-3 chunks retrieved per question to ground GPT generation
- Adaptive difficulty calibration: Junior / Mid / Senior based on resume analysis
- Three interviewer personas: Professional Mentor, Stress Interviewer, Academic Professor
- Real-time answer scoring (1-5) with written GPT feedback per question
- Web Speech API integration: text-to-speech question readout and voice dictation input
- Session UUID recovery: resume any interrupted interview session by pasting its UUID
- Evaluation dashboard: animated SVG score gauge, per-question breakdown, topic coverage
- Offline fallback: keyword-overlap heuristic scoring when OpenAI API is unavailable
- Sample profile mode: run a full demo interview without uploading a PDF

6. API Reference

Method	Endpoint	Description
POST	/sessions	Create a new interview session
POST	/sessions/{id}/resume	Upload and parse candidate resume PDF
POST	/sessions/{id}/mock-resume	Use a demo profile (no PDF needed)
GET	/sessions/{id}/question	Fetch next RAG-grounded question
POST	/sessions/{id}/answer	Submit answer - returns score + feedback
POST	/sessions/{id}/complete	Mark session as complete
GET	/sessions/{id}/summary	Retrieve full evaluation report
GET	/sessions/{id}	Inspect or recover session state
GET	/	Health check endpoint

Interactive API docs available at: <http://localhost:8000/docs>

7. Project Structure

```
interview-ai/
  backend/
    main.py          -- FastAPI application entry point
    db.py            -- SQLAlchemy engine and session factory
    models/          -- ORM models (Session, QAPair)
    routers/
      sessions.py    -- Session creation, resume upload endpoints
      interview.py   -- Question, answer, summary endpoints
    services/
      session_service.py -- Orchestration and business logic
  rag/
    embedder.py      -- PDF ingestion -> FAISS index builder
    retriever.py      -- Semantic similarity search
    generator.py      -- GPT question generation + scoring
  knowledge_base/    -- ML textbook PDFs (not committed to Git)
  faiss_index/       -- Auto-generated vector index files
  requirements.txt
  frontend/
    src/
      App.jsx        -- Application shell, routing, settings modal
      index.css       -- Design system (professional light UI)
      components/
        UploadScreen.jsx -- Role selection & resume upload
```

InterviewScreen.jsx -- Live Q&A with TTS and STT
SummaryScreen.jsx -- Evaluation dashboard
vite.config.js

8. Setup & Running Locally

Prerequisites

- Python 3.10+
- Node.js 18+
- OpenAI API key (optional - local fallback available)

Backend Setup

```
cd backend
pip install -r requirements.txt
# Place ML textbook PDFs in backend/knowledge_base/
python rag/embedder.py          # builds FAISS index from textbooks
# Create .env: OPENAI_API_KEY=sk-... (optional)
python -m uvicorn main:app --reload --port 8000
```

Frontend Setup

```
cd frontend
npm install
npm run dev
# Open http://localhost:5173
```

9. Design Decisions

SQLite over Postgres

Zero configuration required, fits the assignment scope perfectly, and is easily swappable via the SQLAlchemy abstraction layer.

FAISS IndexFlatL2

No cloud dependency, runs entirely locally, and delivers fast nearest-neighbour search on CPU without any server setup.

all-MiniLM-L6-v2 embeddings

80 MB model with strong semantic accuracy. Encodes a 350-word chunk in under 1 second on CPU.

Local fallback system

The system is fully functional without an OpenAI API key. Template questions and

keyword-overlap scoring activate automatically.

Session UUID recovery

Each session is assigned a UUID stored in SQLite, enabling multi-tab usage and interrupted session resumption.

Real textbook grounding

All 6 official assignment textbooks are embedded and indexed. Interview questions reference only textbook content, not generic web data.

10. Known Limitations

- Scanned or image-only PDFs cannot be extracted by PyMuPDF - only PDFs with a text layer are supported.
- The free-tier OpenAI API may throttle requests during large sessions; the local fallback activates automatically.
- Voice dictation (STT) requires Chrome or Edge - Firefox does not support the Web Speech API.
- FAISS IndexFlatL2 is an exact/flat index. For production use, IVF or HNSW would be more scalable.
- The knowledge base covers only the 6 indexed textbooks. Domain knowledge outside these sources is not retrievable.

11. Assignment Compliance Checklist

Requirement	Status
RAG pipeline (FAISS + sentence-transformers + GPT)	Completed
Resume parsing (PyMuPDF skills/tech/level extraction)	Completed
Role-based adaptive questioning	Completed
Real-time answer scoring (1-5) with written feedback	Completed
REST API with documented endpoints	Completed
React frontend with full interview UX	Completed
Session persistence and recovery by UUID	Completed
Evaluation report with analytics and score gauge	Completed
Works offline (no mandatory OpenAI key)	Completed
Official assignment textbooks used as RAG source	Completed - 2,253 chunks indexed

Built for the AI/ML & Backend Engineering Intern Assignment.